

BruFace

MA1 Mechanical Engineering
Vibrations and Acoustics
Academic year 2022-2023

Session 1 : One DOF system

Welcome to the first session

```
In [ ]: # don't worry about this line  
%matplotlib notebook
```

About the Practical sessions

Python and Jupyter

The practical sessions are organized using "Jupyter Notebooks" a tool to program and study results in the Python programming language. It is not our goal to make you ace Python programmers but we do believe it is a useful skill for a modern day engineer. If you are interested to get started with Python there are many resources available online, we recommend: <https://realpython.com/learning-paths/python3-introduction/> (<https://realpython.com/learning-paths/python3-introduction/>).

For these practical sessions we will use these partially filled in notebooks, so you don't need to worry too much about the nuts and bolts of Python and we focus on the Acoustics and Vibrations. We do hope you also get a bit familiar with Python in the progress.

It is important that you run each cell of the notebook. To do so select a cell (it will be highlighted) and press shift+enter on your keyboard or the play button in the menu above.

During the practical session we will make use of three very common library in python. Namely, numpy, scipy and matplotlib

1. NumPy is a fundamental package for scientific computing in Python. It provides powerful matrix manipulation, basic linear algebra and much more
 2. SciPy is a collection of mathematical algorithms and convenience functions built on the NumPy extension of Python.
 3. Matplotlib is a Python 2D plotting library.
-

NB : a package is just a collection of code (module) that you can import and use. If you want to learn more about a package, you shall go and visit the website of the package. E.g. for SciPy: <https://docs.scipy.org/doc/scipy/> (<https://docs.scipy.org/doc/scipy/>).

```
In [ ]: # let's import both numpy and matplotlib.pyplot
# the "as" in the code below is to tell python that if we reference
import numpy as np
import matplotlib.pyplot as plt
```

Then when we want to use a function "foo" in the package numpy we have to type
np.foo(arguments)

Making the exercises

During the exercises you will need to complete some of the cells to make the code work. This will require you to implement some Python code. We kept it fairly simple and we tried to provide all secondary code so you can focus on the assignment at hand.

So let us get started with a first example.

A first example

To get you started let us start with a first exercise. Let us program a function that converts a pulsation ω into a frequency f . We have set you up with the basic structure below.

For this purpose we already set you up with a new function called `convert_pulsation` which takes in one argument `w` which is the pulsation. We also made the function return a value `f`. Can you complete the code (where it says `# your code here`) so the pulsation `w` is converted into a frequency `f` in *Hz*? You should also remove the statement: `raise NotImplementedError`.

TIP: you can access the variable π using the command `np.pi`

```
In [ ]: # cell metadata=autograded-answer
def convert_pulsation(w:float):
    """
    Implement a function that the frequency from a pulsation omega

    Arguments:
    w -- the pulsation

    Returns:
    f -- the frequency in Hz
    """

    ### BEGIN SOLUTION
    f = w/2/np.pi
    ### END SOLUTION

    return f
```

Now that this function is created we can use it as follows : `f =`

`convert_pulsation(w=np.pi)` to convert a pulsation of π rad/s into a frequency.

In the next we can check (`=assert`) whether or not your solution was correct or not. When the cell below runs without errors then your code is working correctly. Note that you can't change this cell yourself.

And, yes you can't see all tests, so we are also checking some other things in the background :-)

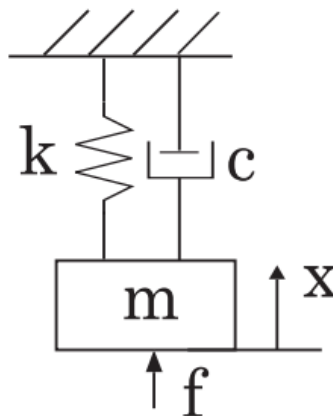
```
In [ ]: # cell metadata=autograder-tests
assert convert_pulsation(np.pi) == 0.5
assert convert_pulsation(2*np.pi) == 1

### BEGIN HIDDEN TESTS
assert convert_pulsation(4*np.pi) == 2
### END HIDDEN TESTS
```

Exercises

Exercise 1 : Fundamental equations of a SDOF system

Consider the following one-degree-of-freedom (1 DOF) system :



Assignment 1.1: Write the equation of motion in the time domain (Using LaTeX) in the cell below:

(If you are not familiar with LaTeX you can use: <https://www.hostmath.com/> and copy the LaTeX below)

Type *Markdown* and LaTeX: α^2

Assignment 1.2: Write the expression to calculate ω_n using the properties of the SDOF system into the function `calcW_n`

```
In [ ]: def calcW_n(k:float,m:float):
        """
        Implement the expression of W_n

        Arguments:
        k, m -- parameters of the SDOF system

        Returns:
        w_n -- the value of w_n, computed using the formula seen in the c
        """

        ### BEGIN SOLUTION
        w_n = np.sqrt(k/m)
        ### END SOLUTION

        return w_n
```

Validation, if the following cell runs without error you have successfully implemented `calcW_n` :

```
In [ ]: # cell metadata=autograder-tests

assert calcW_n(12,3) == 2.0
assert calcW_n(16,4) == 2.0
assert calcW_n(64,4) == 4.0

### BEGIN HIDDEN TESTS
for i in range(5):
    a=np.random.randint(100)
    b=np.random.randint(100)
    assert calcW_n(a,b)==np.sqrt(a/b)
### END HIDDEN TESTS
```

Assignment 1.3: Write the expression to calculate ξ using the properties of the SDOF system into the function `calcXi`

```
In [ ]: def calcXi(m:float, c:float, k:float):
        """
        Implement the expression of Xi

        Arguments:
        m,c,k -- parameter of the SDOF system

        Returns:
        xi -- the value of Xi, computed using the formula seen in the cou
        """

        ### BEGIN SOLUTION
        xi= c/(2*np.sqrt(m*k))
        ### END SOLUTION

        return xi
```

Validation, if the following cell runs without error you have successfully implemented `calcXi` :

```
In [ ]: assert calcXi(1,20,1) == 10.0
assert calcXi(2,20,2) == 5.0
assert calcXi(3,36,3) == 6.0

### BEGIN HIDDEN TESTS
for i in range(5):
    mt=np.random.randint(1,100)
    ct=np.random.randint(1,100)
    kt=np.random.randint(1,100)

    assert calcXi(mt,ct,kt) == ct/(2*np.sqrt(mt*kt))
### END HIDDEN TESTS
```

Assignment 1.4: Write the expression to calculate ω_d , starting from ω_n and ξ , into the function `calcW_d`

```
In [ ]: def calcW_d(w_n:float,xi:float):
        """
        Implement the expression of W_n

        Arguments:
        wn,xi -- properties of the SDOF system

        Returns:
        w_n -- the value of w_n, computed using the formula seen in the c
        """

        ### BEGIN SOLUTION
        w_d = w_n*np.sqrt(1-xi**2)
        ### END SOLUTION
        return w_d
```

Validation, if the following cell runs without error you have successfully implemented `calcW_d`:

```
In [ ]: assert round(calcW_d(1,0.1),3) == 0.995 # The round(results,3) rounds
assert round(calcW_d(2,0.05),3) == 1.997

### BEGIN HIDDEN TESTS
for i in range(5):
    w_n = np.random.randint(1,100)
    xi = np.random.randint(1,50)/100

    assert round(calcW_d(w_n, xi),3) == round(w_n*np.sqrt(1-xi**2),3)
### END HIDDEN TESTS
```

End of Exercise : We have reached the end this first assignment, you have now implemented three functions: ``calcW_n``, ``calcXi`` and ``calcW_d``. These allow us to calculate the properties of any SDOF system.

Exercise 2: Impulse response of a SDOF

Let us put the functions that you wrote in Exercise 1 to good use. Consider a SDOF system, as illustrated in the Figure at the beginning of Exercise 1, with following properties:

- a mass of 1 kg
- a stiffness of 16 N/m
- a damping coefficient of 0.1 Ns/m .

Assignment 2.1: Calculate ω_n , ξ and ω_d for the system described above using the functions made in Exercise 1.

In []:

```
### BEGIN SOLUTION
m = 1
c = 0.1
k = 16

### END SOLUTION

w_n = calcW_n(m=m, k=k)
xi = calcXi(m=m, c=c, k=k)
w_d = calcW_d(w_n, xi)

conclusion = f'The system has an natural frequency of {w_n/2/np.pi:.3f} Hz
print(conclusion)
```

We now have determined the modal parameters for the SDOF system. Let us try to calculate the impulse response of the SDOF system. This impulse response is the behaviour how the system responds to a impulse load.

Assignment 2.2: Implement the impulse response function for SDOF systems in the function `impulse_response`. This is implemented using the equations you built in Exercise 1.

```
In [ ]: def impulse_response(m:float,c:float,k:float,t:np.ndarray):
        """
        Implement the SDOF impulse response

        Arguments:
        m, c and k -- Parameter of the system
                     -- m is in kg
                     -- c is in N.s.m^-1
                     -- k is in N.m^-1
        t          -- Time vector, containing the timesteps

        Returns:
        h -- the value of function h for the timesteps in the Time vector
             expression of the impulse response
        -----
        tips : 1.call the function defined above to compute Xi and Wn and
                save them in a local variable
                2.define a variable wd first
                3.use np.exp and np.sin
        -----
        """

        xi=calcXi(m=m, c=c, k=k)

        w_n=calcW_n(m=m, k=k)

        w_d= calcW_d(w_n, xi)

        ### BEGIN SOLUTION
        h=1/(m*w_d)*np.exp(-xi*w_n*t)*np.sin(w_d*t)
        ### END SOLUTION
        return h
```

Validation, if the following cell runs without error you have successfully implemented `impulse_response` . To test the functions we will use some logical tests. Have a look at the tests below, do you understand their motivation?

```
In [ ]: m = 1
        c = 0.1
        k = 16

        assert impulse_response(m=m, c=c, k=k, t=0) == 0          # at t=0, no response
        assert impulse_response(m=m, c=c, k=k, t=1e8) < 1e-18    # After a long time, response is zero

        # What happens in absence of damping (c=0) ?
        assert not impulse_response(m=m, c=0, k=k, t=1e8) < 1e-18 # In absence of damping, response is not zero
```

Did you understand all of the tests we used above? Maybe it helps to understand them if we plot the first 100 s of the impulse response for the system of interest.

```
In [ ]: t = np.arange(start=0, stop=100, step=0.1) # Defining a time vector with
h = impulse_response(m=1, k=16, c=0.1, t=t) # Calculating the impulse response

# Plotting the result:

plt.figure(figsize=(7,5))
plt.plot(t, h)
plt.xlabel('Time (s)')
plt.ylabel('h(t)')
plt.grid(which='both', linestyle=':')
plt.show()
```

We can clearly see the role of ω_d and ξ in this impulse response $h(t)$. In the figure below we can nicely see how the impulse response is the combination of an undamped sinus (with a frequency ω_d) and an exponential decay (driven by the damping ratio ξ).

```
In [ ]: m = 1
k = 16
c = 0.1

t = np.arange(start=0, stop=100, step=0.1) # Defining a time vector with
h = impulse_response(m=m, k=k, c=c, t=t) # Calculating the impulse response

xi = calcXi(m=m, c=c, k=k)
w_n = calcW_n(m=m, k=k)
w_d = calcW_d(w_n, xi)

# Plotting the result:

plt.figure(figsize=(7,5))
plt.plot(t, np.sin(w_d*t)*np.max(h), color='lightgray', label='$sin(\omega_d t)$')
plt.plot(t, np.exp(-xi*w_n*t)*np.max(h), color='firebrick', label='$e^{-\xi \omega_n t}$')

plt.plot(t, h, label='$h(t)$')

plt.xlabel('Time (s)')
plt.ylabel('h(t)')
plt.xlim([0, 50])

plt.grid(which='both', linestyle=':')

plt.legend()
plt.show()
```

Assignment 2.3:: We have made an interactive tool to plot how the impulse response function varies when you vary m , c , k . Try it out to better understand the interplay of the SDOF properties and its dynamic behaviour. No programming required.

You don't have to understand the following cell. It is a bonus cell for you to have an interactive plot where you can change the system variable and see there impact on the impulse response, ξ , w_n and w_d

```

In [ ]: from ipywidgets import interact, fixed
        from matplotlib.widgets import Slider, Button

        # Define initial parameters
        init_m = 1
        init_k = 16
        init_c = 0.1

        # Create the figure and the line that we will manipulate
        fig, ax = plt.subplots()

        t = np.arange(start=0, stop=100, step=0.1)
        h = impulse_response(m=init_m, k=init_k, c=init_c, t=t)
        line, = plt.plot(t, h, lw=2)
        ax.set_xlabel('Time (s)')
        ax.set_ylabel('h(t)')
        plt.grid(which='both', linestyle=':')

        # adjust the main plot to make room for the sliders
        plt.subplots_adjust(left=0.25, bottom=0.4)

        wn=calcWn(k=init_k,m=init_m)
        Xi=calcXi(c=init_c,k=init_k,m=init_m)
        wd= wn*np.sqrt(1-Xi**2)

        text_wn = plt.text(-43, 0.25, f'$\omega_n$ = {wn:.2f}',
                           style='italic',
                           fontsize = 10,
                           bbox ={'facecolor':'green',
                                   'alpha':0.6, 'pad':2})

        text_Xi = plt.text(-43, 0.2, f'$\xi$ = {Xi*100:.2f}%',
                           style='italic',
                           fontsize = 10,
                           bbox ={'facecolor':'green',
                                   'alpha':0.6, 'pad':2})

        text_wd = plt.text(-43, 0.15, f'$\omega_d$ = {wd:.2f}',
                           style='italic',
                           fontsize = 10,
                           bbox ={'facecolor':'green',
                                   'alpha':0.6, 'pad':2})

        axmass = plt.axes([0.25, 0.1, 0.65, 0.03])
        m_slider = Slider(
            ax=axmass,
            label='Mass (kg)',
            valmin=0.1,
            valmax=30,
            valinit=init_m,
        )

        axstiffness=plt.axes([0.25, 0.15, 0.65, 0.03])
        k_slider = Slider(
            ax=axstiffness,
            label="Stiffness (N/m)",
            valmin=0,
            valmax=20,
            valinit=init_k,

```

```

)
axc = plt.axes([0.25, 0.2, 0.65, 0.03])
c_slider = Slider(
    ax=axc,
    label="damping (N/(m*s))",
    valmin=0.01,
    valmax=1,
    valinit=init_c,
)

# The function to be called anytime a slider's value changes
def update(fixed):
    t = np.arange(start=0, stop=100, step=0.1)
    h = impulse_response(m=m_slider.val, k=k_slider.val, c=c_slider.val)
    line.set_ydata(h)
    Xi = calcXi(c=c_slider.val, k=k_slider.val, m=m_slider.val)
    wn = calcW_n(k=k_slider.val, m=m_slider.val)
    wd = wn*np.sqrt(1-Xi**2)

    text_wn.set_text(f'$\omega_n$ = {convert_pulsation(wn):.2f}Hz')
    text_Xi.set_text(f'$\xi$ = {Xi*100:.2f}%')
    text_wd.set_text(f'$\omega_d$ = {convert_pulsation(wd):.2f}Hz')
    text_wn.set_position((-43, np.max(h)))
    text_Xi.set_position((-43, np.max(h)*(1-0.2)))
    text_wd.set_position((-43, np.max(h)*(1-0.4)))

    ax.relim()
    ax.autoscale_view()
    fig.canvas.draw_idle()

# register the update function with each slider
m_slider.on_changed(update)
k_slider.on_changed(update)
c_slider.on_changed(update)

# Create a `matplotlib.widgets.Button` to reset the sliders to initial
resetax = plt.axes([0.8, 0.025, 0.1, 0.04])
button = Button(resetax, 'Reset', hovercolor='0.975')

def reset(event):
    m_slider.reset()
    k_slider.reset()
    c_slider.reset()
    button.on_clicked(reset)

w = interact(update(fixed))

```

Suggestion : Try to find the parameter that leads to a damping coefficient equal to 100%, observe how w_d becomes nan since the assumption used to derive the expression of w_d no longer holds (damping coef < 1)

End of Exercise : We have reached the end this first assignment, in which you calculated the impulse response of a SDOF system. You should now understand the

interplay ``m``, ``c`` and ``k`` and the modal parameters and how this translates into a different impulse response function.

Exercise 3 : Forced harmonic response.

Let us again consider our SDOF system. But this time try to understand how it would respond to a harmonic force:

$$f(t) = \sin(\omega_f * t).$$

Assignment 3.1: Write a function `harmonic_force` that for a defined driving frequency f (in Hz) calculates the harmonic force $f(t)$ for any timestamp t .

```
In [ ]: def harmonic_force(f:float, t:float):
        """
        Calculate the harmonic force using a driving frequency f for any

        Arguments:
        f -- driving frequency in Hz
        t -- timestamp (in s)

        Returns:
        force -- the harmonic force at timestamp t

        """
        ### BEGIN SOLUTION
        w_f = f*np.pi*2
        force = np.sin(w_f*t)
        ### END SOLUTION
        return force
```

Validation, if the following cell runs without error you have successfully implemented `harmonic_force`

```
In [ ]: assert round(harmonic_force(f=1,t=1),1) == 0
assert round(harmonic_force(f=2,t=1),1) == 0
assert round(harmonic_force(f=0.5,t=1),1) == 0
assert round(harmonic_force(f=0.25,t=1),1) == 1
```

The next step is to calculate the response of our SDOF system $x(t)$ to this harmonic force. To do so we will use Duhamel's integral.

$$x(t) = \int_0^t f(\tau)h(t - \tau)d\tau$$

Or since $h(t) = 0$ and $f(t) = 0$ for all $t < 0$, so we can write it as a convolution:

$$x(t) = \int_{-\infty}^{\infty} f(\tau)h(t - \tau)d\tau$$

$$x(t) = f(t) * h(t)$$

This is quite simple to implement, we use the `np.convolve` command to return the

convolution of the outcome of harmonic_force and impulse_response

```
In [ ]: def harmonic_response(m:float,c:float,k:float,f:np.ndarray,t:np.ndarray)
        """
        Implement the SDOF impulse response

        Arguments:
        m, c and k -- Parameter of the system
                    -- m is in kg
                    -- c is in N.s.m-1
                    -- k is in N.m-1
        t          -- Time vector, containing the timesteps

        Returns:
        h -- the value of function h for the timesteps in the Time vector
            expression of the impulse response
        -----
        tips : 1.call the function defined above to compute Xi and Wn and
                save them in a local variable
                2.define a variable wd first
                3.use np.exp and np.sin
        -----
        """

        f = harmonic_force(f, t)
        h = impulse_response(m=m, c=c, k=k, t=t)

        response = np.convolve(h, f, mode='full') # This is where the magic happens

        return response[:len(f)]
```

Let us have a look at how the harmonic response looks like for our SDOF system with for a harmonic excitation at 0.5Hz.

```
In [ ]: m = 1
        c = 0.1
        k = 16

        t = np.arange(start=0,stop=100,step=0.1)
        w_n=calcW_n(m=m, k=k)

        response = harmonic_response(m=m, c=c, k=k, f=0.05, t=t)

        plt.figure(figsize=(7,5))
        # let's fill our figure with the data we found after using our
        # impulse response data
        plt.plot(t,response)
        # let's add labels to our axes
        plt.xlabel('Time (s)',fontsize=12)
        plt.ylabel('Harmonic response x(t)',fontsize=12)
        plt.grid(which='both', linestyle=':')
        # plt.xlim([0,500])
        plt.show()
```

Remark : In the above figure the transient and the steady state regime can be clearly observed. The first 3 cycles correspond to the transient regime.

Assignment 3.2: We have made an interactive tool to plot how the harmonic response function varies when you vary the driving frequency f . Try it out to better understand the interplay of the SDOF system and the driving frequency. Which types of transient behaviour can you spot? What is the relation with structures natural frequency ω_n ?

In []:

```

# Define initial parameters
m = 1
k = 16
c = 0.1

f_init = 0.64

# Create the figure and the line that we will manipulate
fig, ax = plt.subplots(2,1, sharex=True)

t = np.arange(start=0, stop=100, step=0.02)
f = harmonic_force(f=f_init, t=t)
response = harmonic_response(m=m, c=c, k=k, f=f_init, t=t)
line, = ax[1].plot(t, response, lw=2)
line_f, = ax[0].plot(t, f, lw=2)

ax[0].set_ylabel('f(t)')
ax[0].grid(which='both', linestyle=':')

ax[1].set_xlabel('Time (s)')
ax[1].set_ylabel('x(t)')
ax[1].grid(which='both', linestyle=':')

# adjust the main plot to make room for the sliders
plt.subplots_adjust(left=0.25, bottom=0.3)

wn=calcW_n(k=k,m=m)

text_wn = plt.text(-43, 0.25, f'f = {convert_pulsation(wn):.2f}Hz',
                    style='italic',
                    fontsize = 10,
                    bbox={'facecolor':'green',
                        'alpha':0.6, 'pad':2})

axmass = plt.axes([0.25, 0.1, 0.65, 0.03])
f_slider = Slider(
    ax=axmass,
    label='f (Hz)',
    valmin=0.1,
    valmax=2,
    valinit=init_m,
)

# The function to be called anytime a slider's value changes
def update(fixed):
    response = harmonic_response(m=m, c=c, k=k, f=f_slider.val, t=t)
    f = harmonic_force(f=f_slider.val, t=t)

    ax[1].relim()
    ax[1].autoscale_view()

    line.set_ydata(response)
    line_f.set_ydata(f)

```

```
fig.canvas.draw_idle()

# register the update function with each slider
f_slider.on_changed(update)

# Create a `matplotlib.widgets.Button` to reset the sliders to initial
# resetax = plt.axes([0.8, 0.025, 0.1, 0.04])
button = Button(resetax, 'Reset', hovercolor='0.975')

def reset(event):
    f_slider.reset()
    button.on_clicked(reset)

w = interact(update(fixed))
```

Suggestion : Vary the excitation frequency and observe how the transient regime behaves: In the transient regime, we have the presence of two harmonics: the natural frequency + the excitation frequency, which for certain frequency values leads to the beating phenomenon.

In the previous assignment you could clearly see different types of transient behaviour. After this transient behaviour has passed we can observe the steady state behaviour.

One way to analyse the steady-state behaviour of a system subjected to different harmonic excitations is to work with the frequency domain expression and plotting it in a **Bode diagram**.

Assignment 3.3: Write a function `frequency_response_function` that computes the frequency response function, for harmonic excitations, for any frequency f (in Hz) taking the system properties m , c and k as inputs.

```
In [ ]: def frequency_response_function(m:float,c:float,k:float,f:np.ndarray)
        """
        Implement the SDOF expression of the harmonic forced response

        Arguments:
        m, c and k -- Parameter of the system
                    -- m is in kg
                    -- c is in N.s.m^-1
                    -- k is in N.m^-1
        f -- vector of frequencies (Hz)

        Returns:
        H -- a vector of the value of the function H, computed using
             expression of the expression of the harmonic forced response
        -----
        tips : In python the complex i can be expressed using the letter
        -----
        """
        ### BEGIN SOLUTION
        w = np.pi*2*f
        wn = calcW_n(m=m,k=k)
        xi = calcXi(m=m,c=c,k=k)
        H=1/(m*(wn**2-w**2+2j*xi*w*wn))
        ### END SOLUTION
        return H
```

Validation, if the following cell runs without error you have successfully implemented frequency_response_function :

```
In [ ]: assert isinstance(frequency_response_function(m=1, c=0.1, k=16, f=10), complex)
        assert round(abs(frequency_response_function(m=1, c=0.1, k=16, f=0.628)),2) == 0.62
        assert round(abs(frequency_response_function(m=1, c=0.1, k=16, f=0.628)),2) == 0.62
```

As you can tell from the tests above the outcome of the frequency response function is a complex number. Therefore it carries information about both the **Amplitude** and **phase**. A Bode diagram visualizes exactly these elements of a frequency response function. Let us draw a Bode diagram of our SDOF system for the frequency range of 0 to 2 Hz.

We already provided a function `bode_plot` that nicely draws the Bode plot.

Assignment 3.4: Use the provided function `bode_plot` to draw the Frequency response function (in the range from 0 to 2 Hz) for a system with :

- a mass of 1 kg
- a stiffness of 16 N/m
- a damping coefficient of 0.1 N.s/m .

```

In [ ]: def bode_plot(f, H):
        """
        Draw a bode plot for the frequency response function H and corres

        Arguments:
        H -- a vector of the value of the function H, computed using
             expression of the expression of the harmonic forced respo
        f -- vector of frequencies (Hz)
        """

        fig,ax= plt.subplots(nrows=2,figsize=(8,6))
        ax[0].plot(f,np.abs(H))
        ax[0].set_yscale('log')
        ax[0].set_ylabel('Amplitude ')
        ax[0].set_xlabel('Frequency (Hz)')
        ax[0].grid(which='both', linestyle=':')

        ax[1].plot(f,np.angle(H,deg=True))
        ax[1].set_xlabel('Frequency (Hz)')
        ax[1].set_ylabel('Phase (deg)')
        ax[1].grid(which='both', linestyle=':')

        ### BEGIN SOLUTION
        f= np.arange(0,2,0.002)
        H = frequency_response_function(m=m,c=c,k=k,f=f)
        bode_plot(f,H)
        ### END SOLUTION

```

So what does this mean? To illustrate how this relates to the system response to a harmonic force (of a fixed amplitude but variable frequency) we made a small demonstration. Note that despite we are not changing the input amplitude that the output amplitude does vary! Can you maximize the response amplitude?

In []:

```

# Define initial parameters
m = 1
k = 16
c = 0.1

f_init = 0.64

# Create the figure and the line that we will manipulate
fig, axes = plt.subplots(1,2)

t = np.arange(start=0,stop=100,step=0.1)
response = harmonic_response(m=m, c=c, k=k, f=f_init, t=t)

force = harmonic_force(f=f_init, t=t)

f = np.arange(0,2,0.002)
H = frequency_response_function(m=m,c=c,k=k,f=f)

line, = axes[0].plot(t,response, lw=2)

axes[0].set_xlabel('Time (s)')
axes[0].set_ylabel('x(t)')
axes[0].grid(which='both', linestyle=':')

response = harmonic_response(m=m, c=c, k=k, f=f_init, t=t)
axes[1].semilogy(f,np.abs(H))

red_dot, = axes[1].semilogy(
    f_init,
    np.abs(frequency_response_function(m=m,c=c,k=k,f=f_init)),
    marker='.',
    markersize=10,
    color='firebrick')

axes[1].set_xlabel('Frequency (Hz)')
axes[1].set_ylabel('|H(f)|')
axes[1].grid(which='both', linestyle=':')

# adjust the main plot to make room for the sliders
plt.subplots_adjust(left=0.1, bottom=0.4)

wn=calcW_n(k=k,m=m)

text_wn = plt.text(-43, 0.25, f'$\omega_n$ = {convert_pulsation(wn):.2f}',
    style='italic',
    fontsize = 10,
    bbox ={'facecolor':'green',
           'alpha':0.6, 'pad':2})

axmass = plt.axes([0.25, 0.1, 0.65, 0.03])
f_slider = Slider(
    ax=axmass,
    label='f (Hz)',
    valmin=0.1,
    valmax=2,

```

```
        valinit=init_m,
    )

# The function to be called anytime a slider's value changes
def update(fixed):
    response = harmonic_response(m=m, c=c, k=k, f=f_slider.val, t=t)
    val = frequency_response_function(m=m, c=c, k=k, f=f_slider.val)
    line.set_ydata(response)
    red_dot.set_ydata(np.abs(val))
    red_dot.set_xdata(f_slider.val)
    axes[0].set_xlim([80, 100])
    axes[0].set_ylim([-25, 25])
    fig.canvas.draw_idle()

# register the update function with each slider
f_slider.on_changed(update)

# Create a `matplotlib.widgets.Button` to reset the sliders to initial
# resetax = plt.axes([0.8, 0.025, 0.1, 0.04])
button = Button(resetax, 'Reset', hovercolor='0.975')

def reset(event):
    f_slider.reset()
    button.on_clicked(reset)

w = interact(update(fixed))
```

So what about the phase?

In []:

```

# Define initial parameters
m = 1
k = 16
c = 0.1

f_init = 0.64

# Create the figure and the line that we will manipulate
fig, axes = plt.subplots(1,2)

t = np.arange(start=0,stop=100,step=0.1)
response = harmonic_response(m=m, c=c, k=k, f=f_init, t=t)
force = harmonic_force(f=f_init, t=t)
f = np.arange(0,2,0.002)

H = frequency_response_function(m=m,c=c,k=k,f=f)

line, = axes[0].plot(t,response, lw=2)
line_f, = axes[0].plot(t,force, lw=2)

axes[0].set_xlabel('Time (s)')
axes[0].set_ylabel('x(t)')
axes[0].grid(which='both', linestyle=':')
axes[0].legend(['$x(t)$', '$f(t)$'])
response = harmonic_response(m=m, c=c, k=k, f=f_init, t=t)
axes[1].plot(f,np.angle(H,deg=True))

red_dot, = axes[1].plot(
    f_init,
    np.angle(frequency_response_function(m=m,c=c,k=k,f=f_init), deg=1),
    marker='.',
    markersize=10,
    color='firebrick')

axes[1].set_xlabel('Frequency (Hz)')
axes[1].set_ylabel('angle(H(f))')
axes[1].grid(which='both', linestyle=':')

# adjust the main plot to make room for the sliders
plt.subplots_adjust(left=0.1, bottom=0.4)

wn=calcW_n(k=k,m=m)

text_wn = plt.text(-43, 0.25, f'$\omega_n$ = {convert_pulsation(wn):.2f}',
    style='italic',
    fontsize = 10,
    bbox={'facecolor':'green',
        'alpha':0.6, 'pad':2})

axmass = plt.axes([0.25, 0.1, 0.65, 0.03])
f_slider = Slider(
    ax=axmass,
    label='f (Hz)',
    valmin=0.1,
    valmax=2,
    valinit=init_m,

```

```

)

# The function to be called anytime a slider's value changes
def update(fixed):
    response = harmonic_response(m=m, c=c, k=k, f=f_slider.val, t=t)
    val = frequency_response_function(m=m, c=c, k=k, f=f_slider.val)
    force = harmonic_force(f=f_slider.val, t=t)

    line.set_ydata(response)
    line_f.set_ydata(force)
    red_dot.set_ydata(np.angle(val, deg=True))
    red_dot.set_xdata(f_slider.val)
    axes[0].set_xlim([80, 100])
    axes[0].set_ylim([-25, 25])
    fig.canvas.draw_idle()

# register the update function with each slider
f_slider.on_changed(update)

# Create a `matplotlib.widgets.Button` to reset the sliders to initial
# resetax = plt.axes([0.8, 0.025, 0.1, 0.04])
button = Button(resetax, 'Reset', hovercolor='0.975')

def reset(event):
    f_slider.reset()
    button.on_clicked(reset)

w = interact(update(fixed))

```

For our final assignment we will have a look at how the Frequency response function varies for different systems.

In []:

```

# Define initial parameters
init_m = 1
init_k = 16
init_c = 0.1

# Create the figure and the line that we will manipulate
fig, axes = plt.subplots(1,2)

f= np.arange(0,2,0.002)
H = frequency_response_function(m=init_m,k=init_k,c=init_c,f=f)

wn=calcW_n(k=init_k,m=init_m)
axes[0].semilogy(f,np.abs(H), lw=2, color='lightgray')

line, = axes[0].semilogy(f,np.abs(H), lw=2, color='steelblue')

axes[0].set_xlabel('Frequency (Hz)')
axes[0].set_ylabel('|H(f)|')
axes[0].grid(which='both', linestyle=':')

axes[1].plot(f,np.angle(H,deg=True), color='lightgray')

line2, = axes[1].plot(f,np.angle(H,deg=True), color='steelblue')

axes[1].set_xlabel('Frequency (Hz)')
axes[1].set_ylabel('angle(H(f))')
axes[1].grid(which='both', linestyle=':')

linev1 = axes[0].axvline(convert_pulsation(wn), color='firebrick', lw=2)
linev2 = axes[1].axvline(convert_pulsation(wn), color='firebrick', lw=2)

# adjust the main plot to make room for the sliders
plt.subplots_adjust(left=0.25, bottom=0.4)

wn=calcW_n(k=init_k,m=init_m)
Xi=calcXi(c=init_c,k=init_k,m=init_m)
wd= wn*np.sqrt(1-Xi**2)

text_wn = plt.text(-43, 0.25, f'$\omega_n$ = {wn:.2f}',
                  style='italic',
                  fontsize = 10,
                  bbox={'facecolor':'green',
                        'alpha':0.6, 'pad':2})

text_Xi = plt.text(-43, 0.2, f'$\xi$ = {Xi*100:.2f}%',
                  style='italic',
                  fontsize = 10,
                  bbox={'facecolor':'green',
                        'alpha':0.6, 'pad':2})

text_wd = plt.text(-43, 0.15, f'$\omega_d$ = {wd:.2f}',
                  style='italic',
                  fontsize = 10,
                  bbox={'facecolor':'green',
                        'alpha':0.6, 'pad':2})

axmass = plt.axes([0.25, 0.1, 0.65, 0.03])
m_slider = Slider(

```

```

        ax=axmass,
        label='Mass (kg)',
        valmin=0.1,
        valmax=30,
        valinit=init_m,
    )
    axstiffness=plt.axes([0.25, 0.15, 0.65, 0.03])
    k_slider = Slider(
        ax=axstiffness,
        label="Stiffness (N/m)",
        valmin=0,
        valmax=20,
        valinit=init_k,
    )
    axc = plt.axes([0.25, 0.2, 0.65, 0.03])
    c_slider = Slider(
        ax=axc,
        label="damping (N/(m*s))",
        valmin=0.01,
        valmax=1,
        valinit=init_c,
    )

# The function to be called anytime a slider's value changes
def update(fixed):
    h =frequency_response_function(m=m_slider.val,k=k_slider.val,c=c_slider.val)

    line.set_ydata(np.abs(h))
    line2.set_ydata(np.angle(h, deg=True))

    Xi=calcXi(c=c_slider.val,k=k_slider.val,m=m_slider.val)
    wn=calcW_n(k=k_slider.val,m=m_slider.val)
    wd= calcW_d(xi=Xi, w_n=wn)

    linev2.set_xdata(convert_pulsation(wd))
    linev1.set_xdata(convert_pulsation(wd))

    text_wn.set_text(f'$\omega_n$ = {convert_pulsation(wn):.2f}Hz')
    text_Xi.set_text(f'$\xi$ = {Xi*100:.2f}%')
    text_wd.set_text(f'$\omega_d$ = {convert_pulsation(wd):.2f}Hz')

    fig.canvas.draw_idle()

# register the update function with each slider
m_slider.on_changed(update)
k_slider.on_changed(update)
c_slider.on_changed(update)

# Create a `matplotlib.widgets.Button` to reset the sliders to initial values
resetax = plt.axes([0.8, 0.025, 0.1, 0.04])
button = Button(resetax, 'Reset', hovercolor='0.975')

def reset(event):
    m_slider.reset()

```

```
k_slider.reset()  
c_slider.reset()  
button.on_clicked(reset)  
w = interact(update(fixed))
```

Remark : Notice how, when you increase the damping, the amplitude of the resonance decreases. However, for frequencies far from the resonance frequency, the transfer function does not change much even for a high damping.